

AI Dungeon Master for Casual Tabletop RPG Play

ภณลภัส สุทธิมาลา 65340500046

อาจารย์ที่ปรึกษา

ผศ.ดร. สุริยา นัฏสูภักพงศ

ดร. ไพสิฐ ขันอาสา

Phase 1: The Problem & Scope

What is D&D?

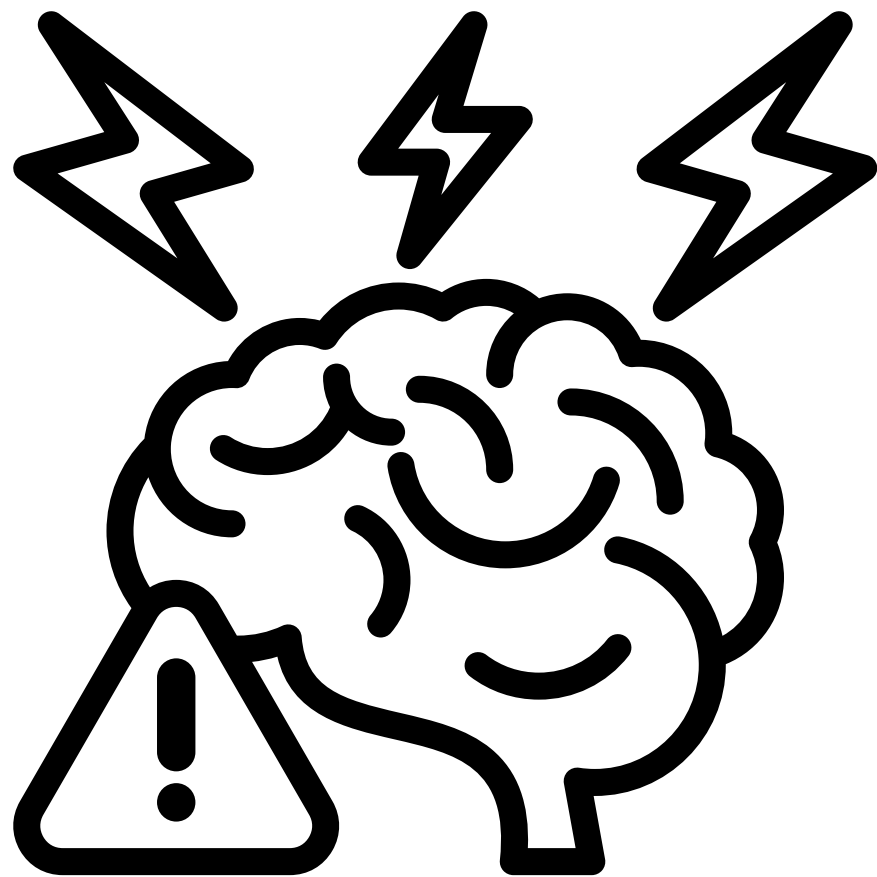


D&D = Analog RPG running on a Human Server

- **Concept: Interactive Storytelling running on a "Human Server"**
- **The Architecture:**
 - **Client: Players send commands.**
 - **Server (DM): Processes logic & generates output.**
- **The DM Workload (Processing Unit):**
 - **As a Narrator: Storytelling & Improvise.**
 - **As a Game Engine: Math, Probability & Rules.**



The Pain Point (The Bottleneck)



The Problem: "Human Server" Overload

1. Processing Overhead:

- Must memorize 100+ pages of Rulebooks.
- Real-time calculation of Stats & Dice Probabilities.

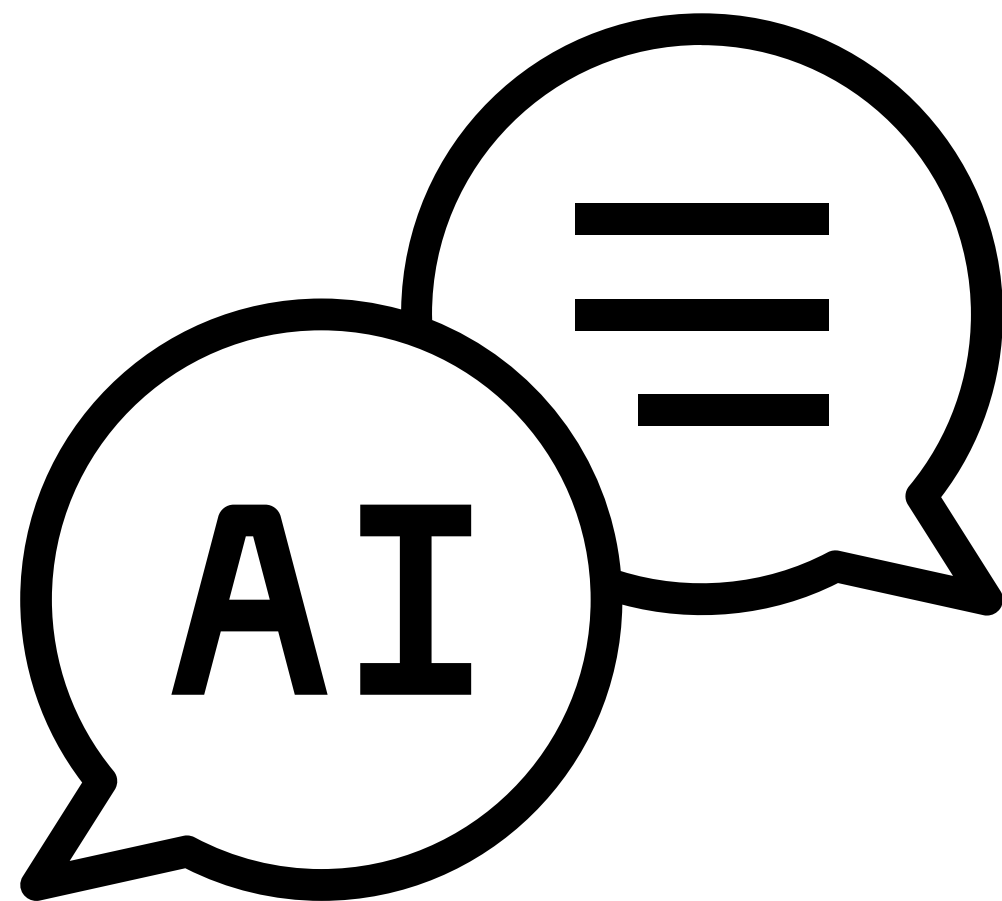
2. High Latency (Preparation):

- World-building takes 4-6 hours per session.
- Imbalanced Ratio: Prep Time > Play Time.

3. System Failure:

- Result: "DM Scarcity" (Critical lack of Game Masters).
- Casual players face "System Lockout" (Access Denied).

The Failure of "Monolithic LLM" (Why not just ChatGPT?)



Just Prompt Engineering?

1. The Hypothesis:

- **Idea:** "Why not just prompt ChatGPT with the Rulebook?"
- **Verdict:** Monolithic LLM Architecture Fails.
 - (Ref: Triyason 2023, Sakellaridis 2024)

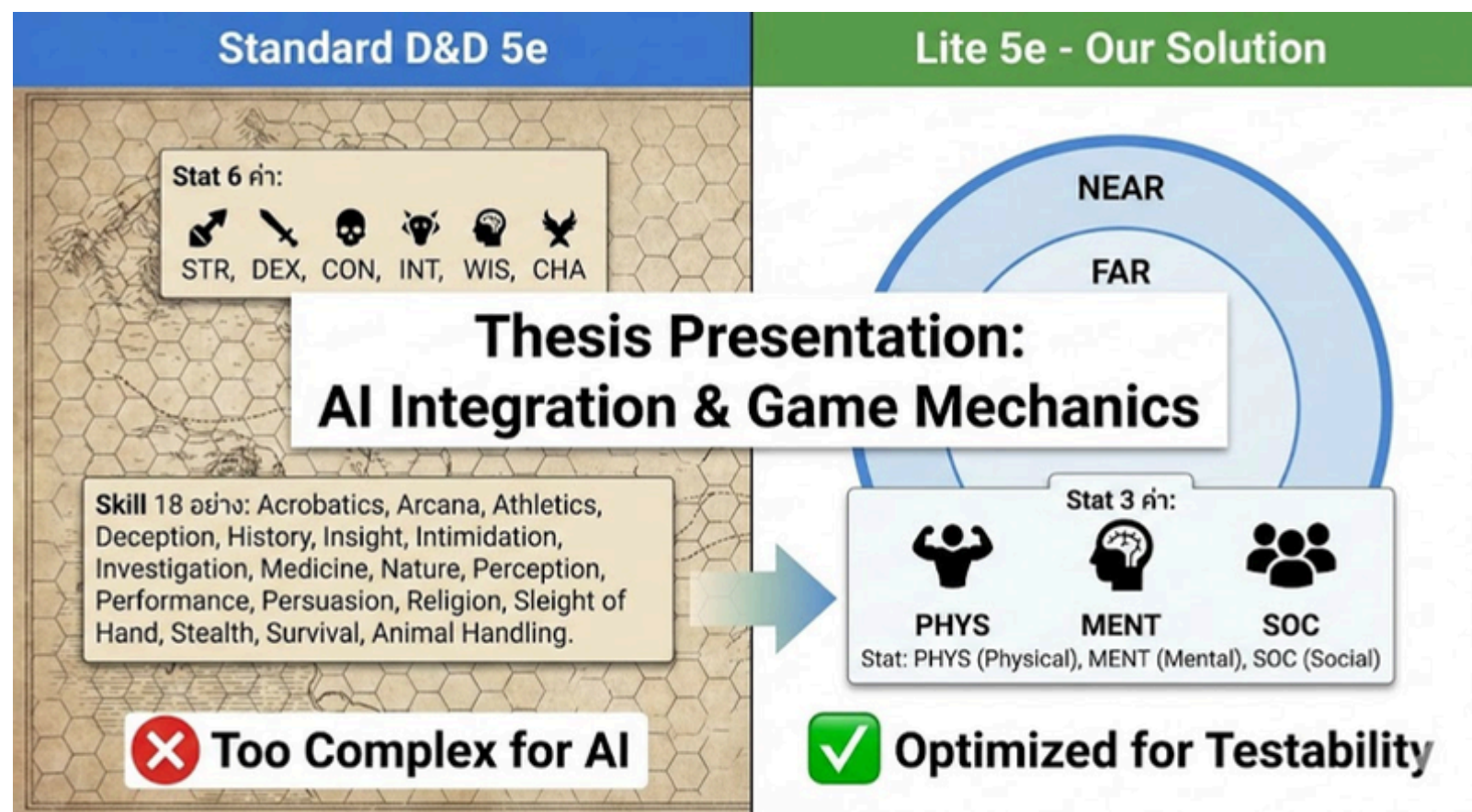
2. The Critical Bugs:

- **Hallucination:** High linguistic capability, but prone to Math/Logic Errors.
- **Statelessness:** Lack of long-term memory leads to State Inconsistency.
- **Railroading:** Forces players into a fixed narrative -> Loss of Player Agency.

3. Conclusion:

- "An LLM alone is Insufficient for the role of Dungeon Master."

Scope Optimization (Lite 5e)



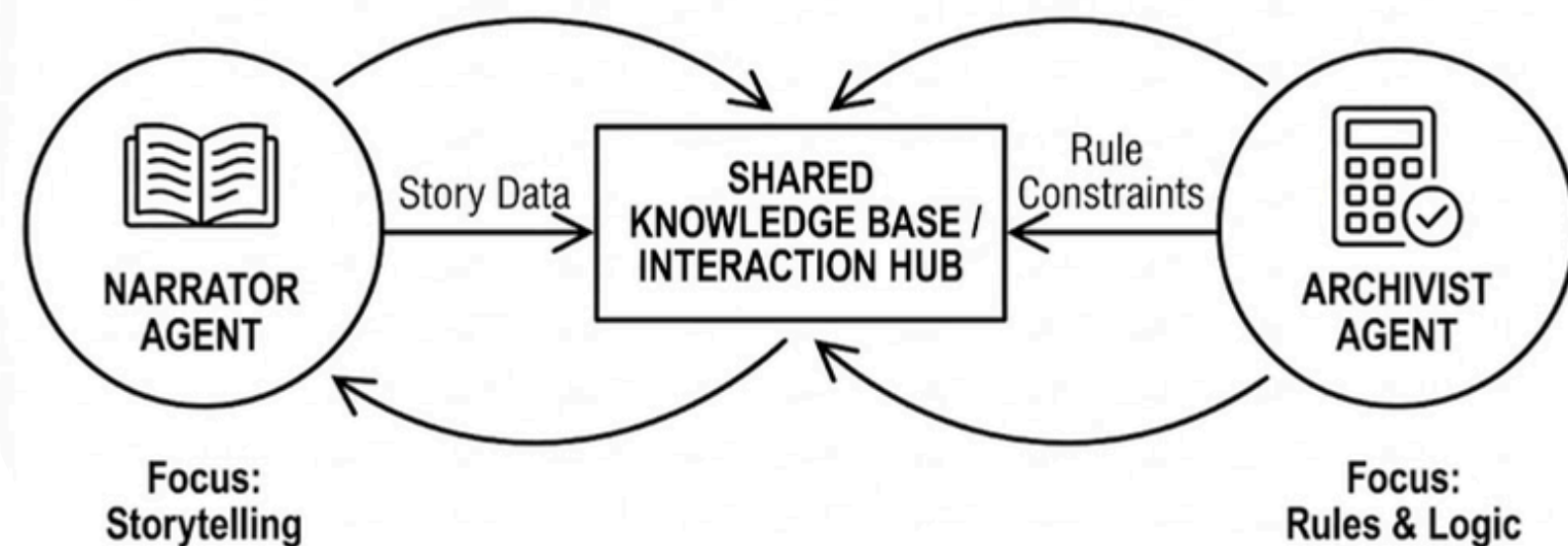
Goal: Create a "Controlled Environment" for AI verification.

- **1. Map System:**
 - **Standard:** 5ft. Grid (Complex calculation).
 - **Lite 5e:** Zone-based (Near/Mid/Far) -> Easier for AI to logic.
- **2. Stat System:**
 - **Standard:** 6 Stats + 18 Skills.
 - **Lite 5e:** 3 Core Stats (PHYS, MENT, SOC) -> Reduces hallucination.
- **3. Result:**
 - Makes the system "Testable" & "Casual-friendly".

Phase 2: The Research & Solution

Insight 1 - The Concept (Jørgensen et al.)

(Ref: Jørgensen et al., 2024)



Insight 1: Multi-Agent Architecture

Reference: Jørgensen et al. (2024) - "ChatRPG"

1. The Finding :

- Used a **"Multi-Agent System"** to split the DM into specific roles:
 - **Narrator Agent:** Focuses only on storytelling.
 - **Archivist Agent:** Focuses on rules & state tracking.

2. The Result :

- Proved that **"Separation of Concerns"** significantly improves consistency.
- Single LLM gets confused, but specialized agents remain stable.

3. Application to our Project :

- We adopt this **"Split-Brain Concept"**: We must separate the Creative Layer from the Logic Layer.

Insight 2 - The Method (Song et al.)

Insight 2: Tool-Assisted AI

Reference: Song et al. (2024)

1. The Finding (สิ่งที่ค้นพบ):

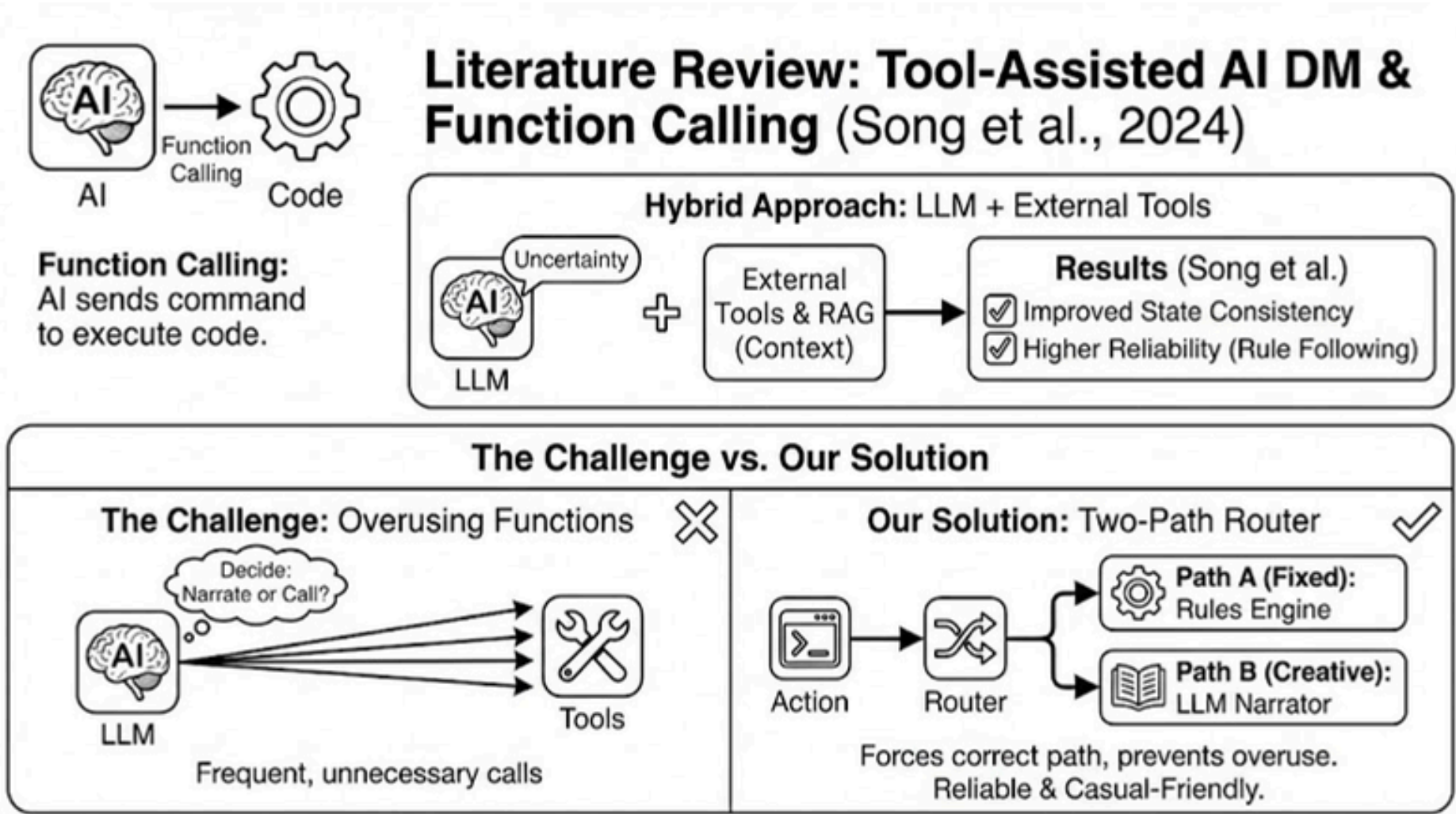
- Integrated "Function Calling" (External Tools) for game mechanics.
- The AI does NOT calculate math; it calls a Code Function instead.

2. The Result (ผลลัพธ์):

- Solved Hallucination in dice rolls and state updates.
- Achieved higher Reliability compared to pure LLM.

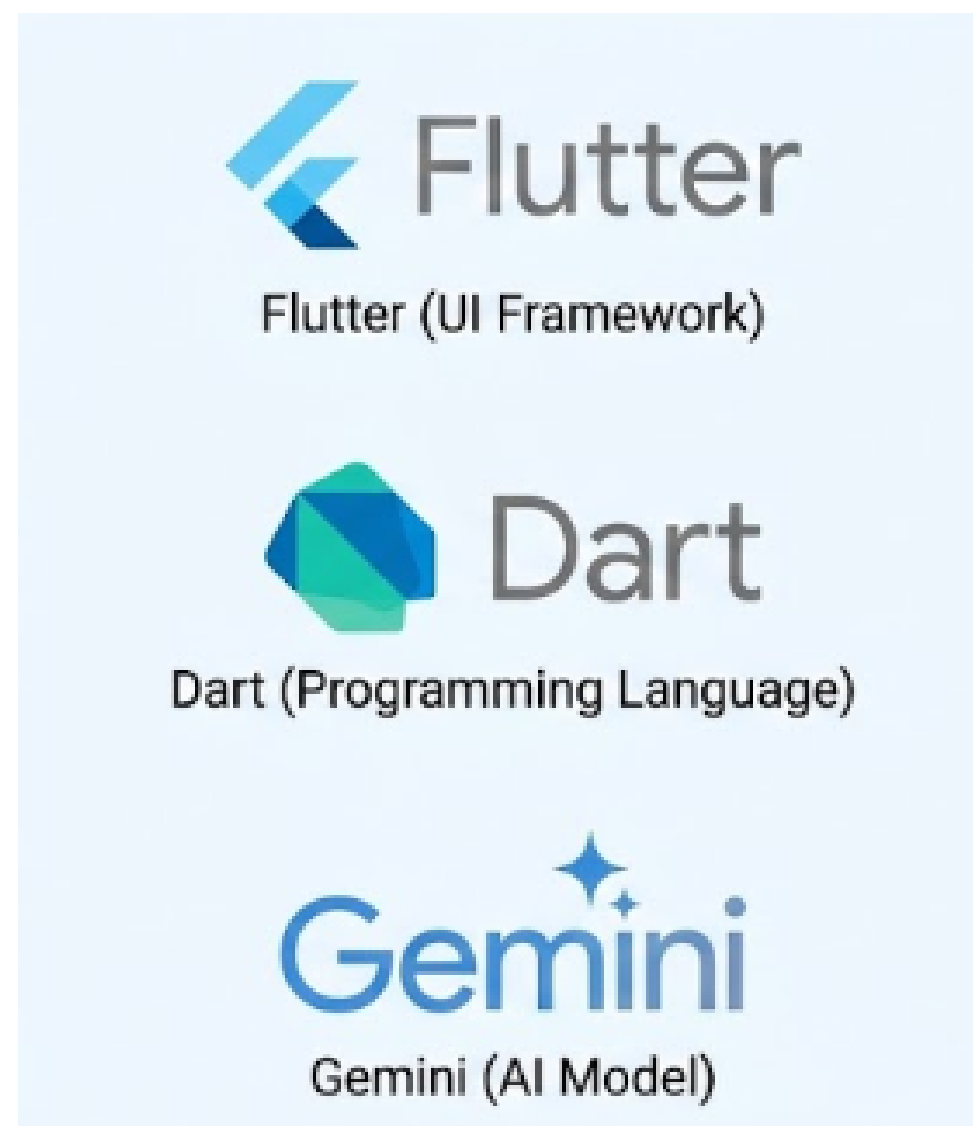
3. Conclusion: The "Mix & Match" Strategy:

- We combine Jørgensen's "Separation" + Song's "Tools".
- **Result:** A Hybrid Architecture where Router assigns tasks to either AI or Code.



Phase 3: The Design Blueprint

Core Philosophy & Tech Stack



The Tech Stack

System Foundation

1. The Tech Stack:

- **Logic:** Developed in Dart using Pure Functions for stability.
- **UI:** Built with Flutter for a Single Codebase (Cross-platform).

2. Architecture Philosophy:

- We designed a "Stateless Machine". The Engine calculates but never stores state locally to prevent bugs.

3. Single Source of Truth:

- All data is stored in one GameState Object to ensure consistency.

4. Deterministic Logic:

- **Rule:** "AI never does Math." All calculations (HP, Dice, Damage) are handled 100% by Code.

AI Model Selection (Why Gemini?)

Comparison Table

เกณฑ์ (Criteria)	Gemini 2.5 Flash-Lite (ตัวเลือกของเรา)	GPT-5 nano (คู่แข่ง สายประหยัด)	Claude 3 Haiku 4.5 (คู่แข่งสายเร็ว)
1. Cost (ราคา/Token) (Input/Output ต่อ 1M) (Goal: "Casual" ต้องถูก)	\$0.10 / \$0.40	\$0.05 / \$0.40	\$1.00 / \$5.00
2. Speed (ความเร็ว/Latency) (Goal: "Portable" ต้องเร็ว)	เร็วที่สุด (Ultra Fast)	เร็วมาก (Very fast)	เร็วที่สุด (Fastest)
3. Context Window (หน่วย ความจำ) (Goal: D&D เรื่องยาว)	1 ล้าน Tokens	400K Tokens	(ไม่ระบุชัด แต่ Sonnet/Opus มี 1M+)
4. Function Calling (Goal: รองรับ "Two-Path")	รองรับ (Supported)	รองรับ (Supported)	รองรับ (Supported)
5. Quality (คุณภาพ) (Goal: ดีพอสำหรับ Lite 5e)	ดี (Optimized for cost- efficiency)	พอใช้ (Fastest, cheapest... for summarization)	ดีมาก (Fastest, cost- efficient)

 (Cost),  (Speed),  (Memory)

Why Gemini 2.5 Flash-Lite?

1. The Selection Criteria:

- We needed a model that balances Cost, Speed, and Features for a casual app.

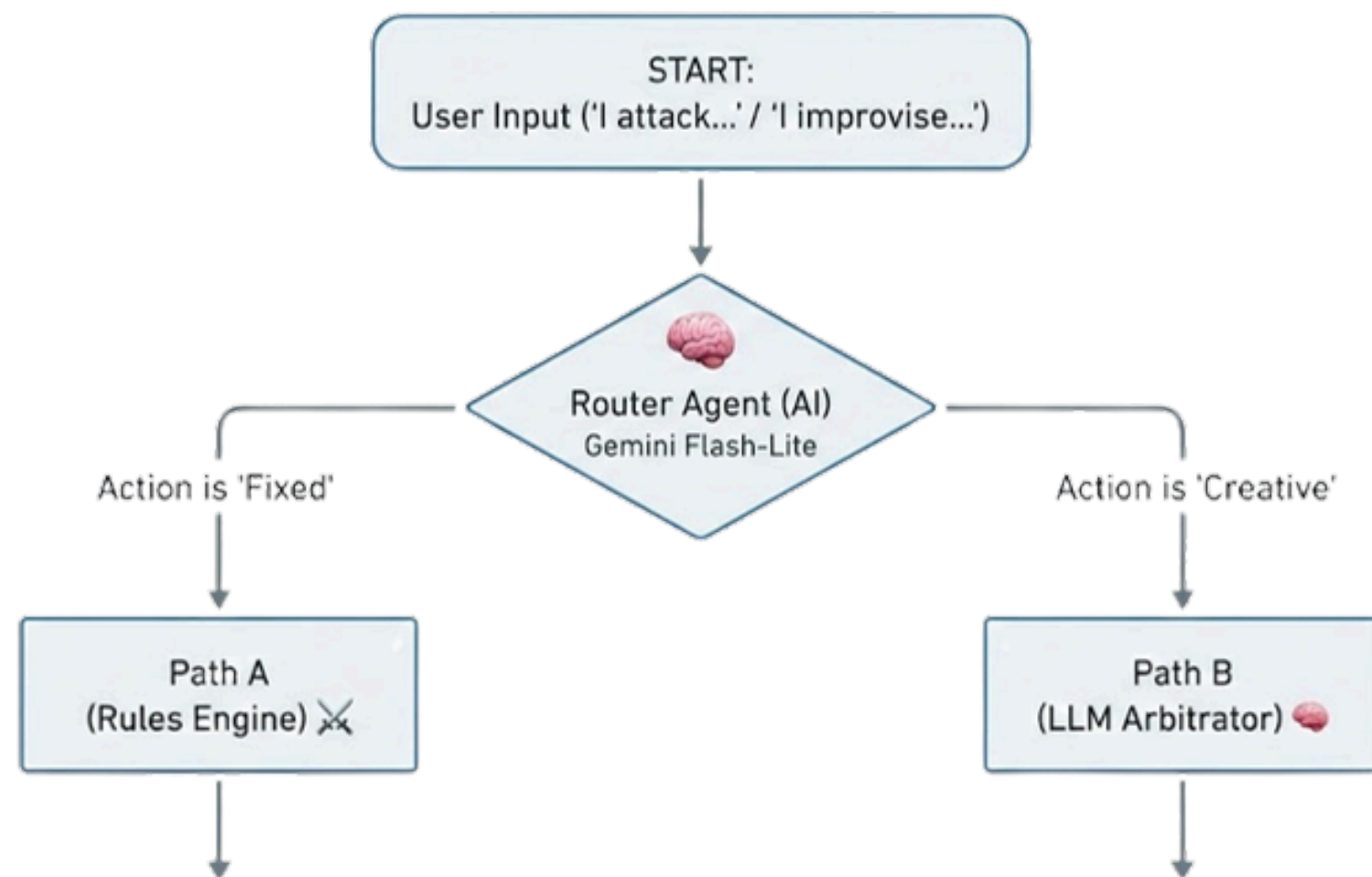
2. The "Reasonable" Choice:

- **Cost & Speed:** High throughput with low latency, essential for real-time play.
- **Feature Set:** Fully supports Function Calling, critical for our Hybrid Architecture.

3. The Winning Factor:

- 1 Million Token Context Window:
- Superior to competitors (typ. 128k).
- Necessary for remembering long-term history in D&D sessions.

The Router Logic (The Brain)



The Router Logic: "Hybrid-Arbitrated"

1. The Decision Process:

- The Router is a specialized AI Agent (Gemini Flash-Lite) acting as a Classifier.
- It analyzes the user's Intent (Fixed vs. Creative) in real-time.

2. Path A: Fixed Actions (Rule-First):

- **Trigger:** Commands like "Attack", "Cast Spell", or "Move".
- **Action:** Sent directly to the Rules Engine (Code).
- **Result:** 100% Deterministic and Accurate.

3. Path B: Creative Actions (LLM-First):

- **Trigger:** Complex actions like "Tie a rope to trip the goblin."
- **Action:** Sent to the LLM Arbitrator to decide the Difficulty Class (DC).
- **Result:** Flexible creativity grounded by game mechanics.

Data Strategy (TOON & RAG)

JSON

```
{
  "players": [
    {
      "name": "Valen",
      "hp": 18,
      "class": "Fighter" }
  ]
}
```

TOON

```
players[1]{name, hp, class}:
Valen, 18, Fighter
```

Reduced ~40% Tokens

Data Strategy: Efficiency & Context

1. Hybrid Data Models:

- **Internal:** Used Dart Classes for fast app processing and UI updates.
- **External:** Converted data into TOON Format (Token-Oriented Object Notation) for AI communication.

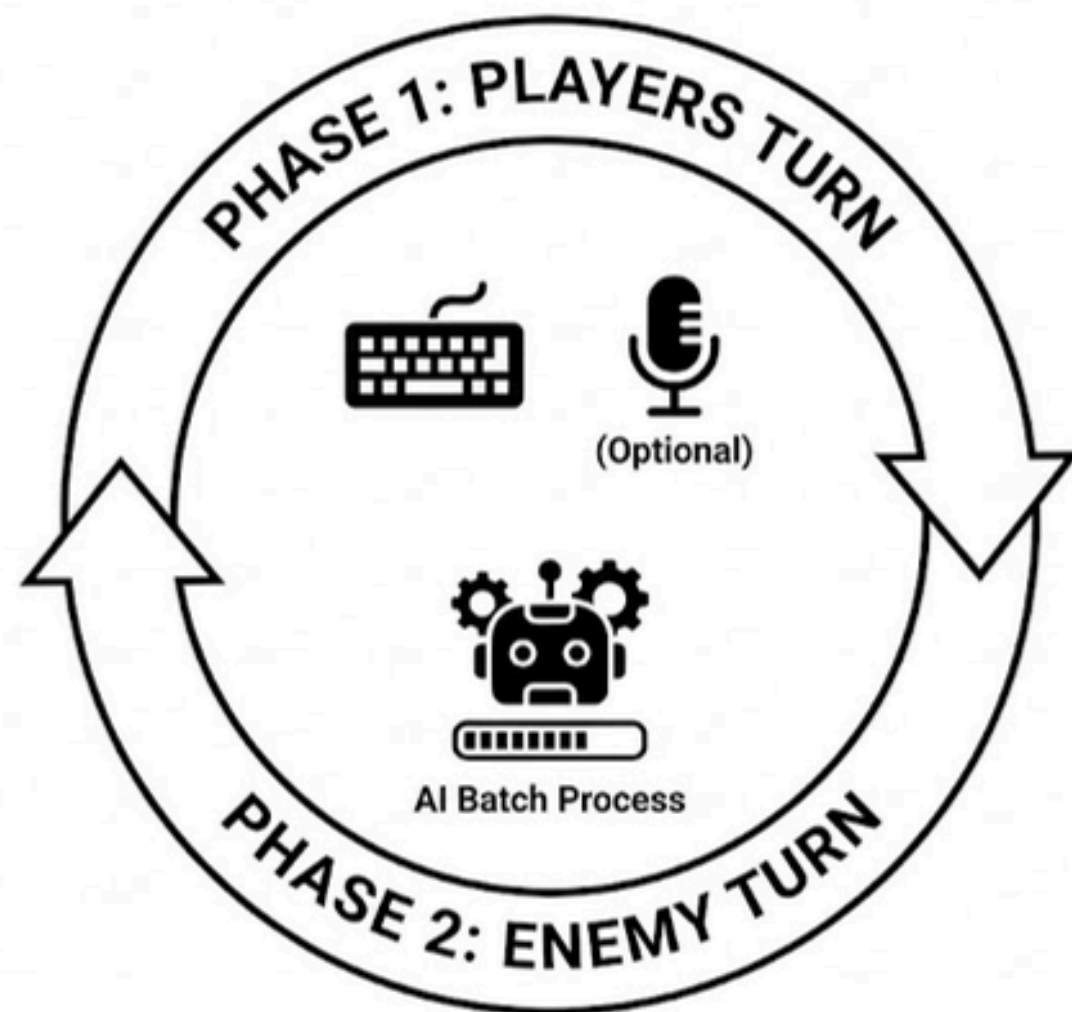
2. Why TOON?

- **Optimized for LLMs:** Reduces Token usage by ~40% compared to JSON.
- **Benefit:** Significantly lowers Cost and Latency.

3. RAG Strategy (Retrieval-Augmented Generation):

- **We inject a Context Stack into every prompt:**
 - a. Current State (Mechanics)
 - b. Narrative Goals (Story)
 - c. Action History (Memory)
- **Goal:** Prevents the AI from "forgetting" or hallucinating.

The Combat Loop & UX



Combat Loop: "Side Initiative"

1. The UX Challenge:

- Playing on a Single Device is disruptive if we have to pass the screen back and forth constantly.

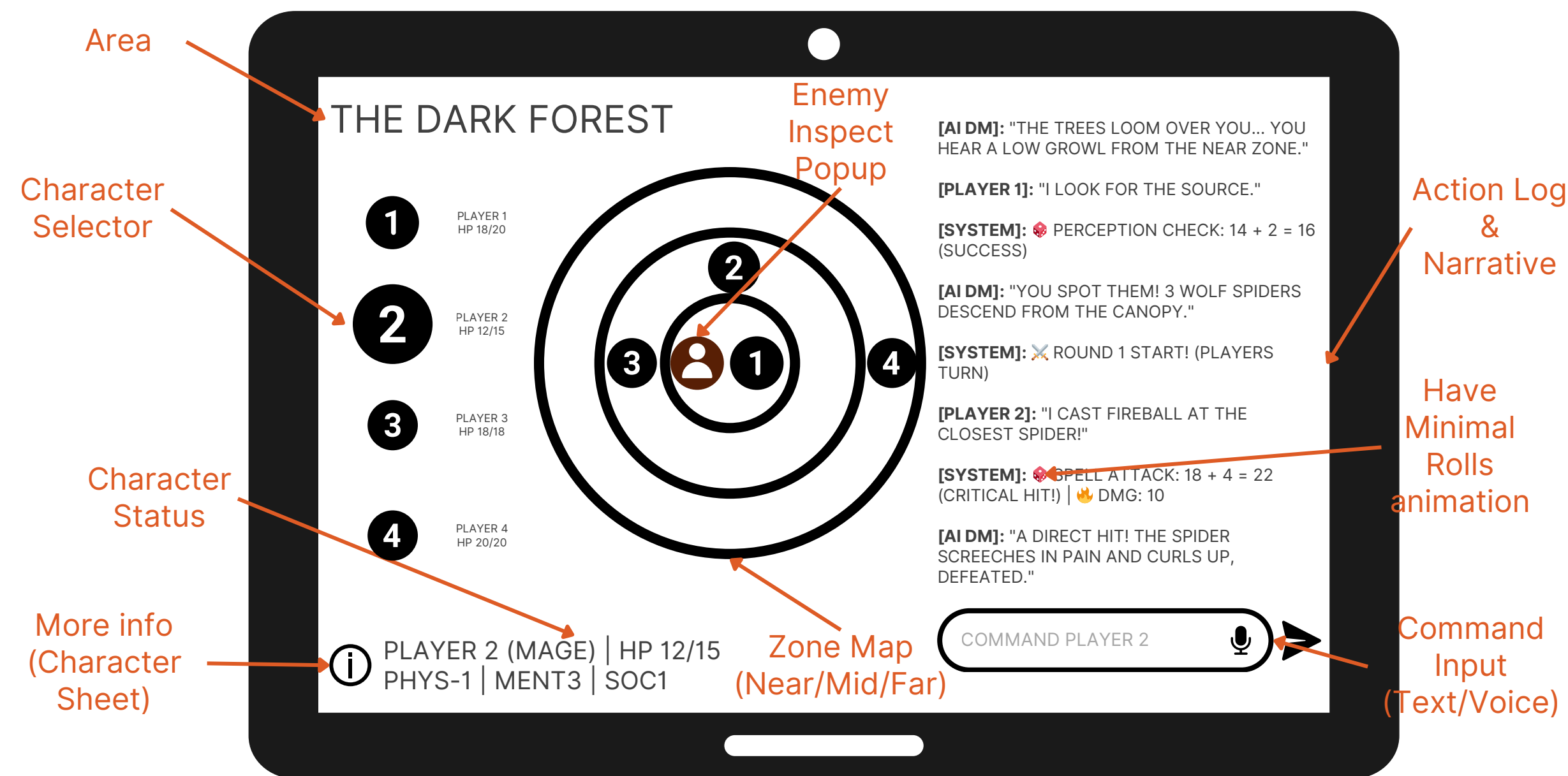
2. The Solution (Side Initiative):

- **Phase 1 (Player Turn):** All players discuss and input actions freely.
 - **Note:** Supports Text Input (Primary) and Voice ASR (Optional Feature).
- **Phase 2 (Enemy Turn):** The AI calculates actions for ALL enemies at once and generates a single narrative summary.

3. The Result:

- Creates smooth Flow Management and speeds up gameplay significantly on mobile devices.

UI Design Strategy (Main Screen)



UI Design: Meeting User Requirements

1. The Requirement:

- **Req 1.5:** Support "Single Device" play (multiple players sharing one screen).
- **Req 1.4:** Simplify visualization for "Lite 5e" (Casual-friendly).

2. The Solution (Main Dashboard):

- **Character Selector:** Vertical bar for quick-switching between players without changing screens.
- **Zone Map:** Replaced complex grids with a 3-Zone Ring (Near/Mid/Far) for easy visibility on mobile.
- **Action Log:** Separated System Logs (Math) from Narrative (Story) for clarity.

Interaction Design (Details & Popups)

Character Sheet Overlay



Enemy Inspect Popup

Strategic & Immersive

1. Character Sheet (Party Overview):

- **Design:** An overlay showing all 4 characters side-by-side.
- **Goal:** Meets the "Single Device" need. Players can compare Stats & Abilities instantly to decide who should act next, without passing the device around.

2. Enemy Inspection (Hidden Info):

- **Design:** Hides raw numbers (Exact HP/AC).
- **Why?** To mimic the Real D&D Experience. A DM never tells you "The goblin has 5 HP left."
- **Feedback:** Shows status qualitatively (e.g., "Healthy", "Injured", "Critical") to maintain tension and immersion.

Phase 4: The Implementation & Proof

Project Timeline (Gantt Chart)

ลำดับ	แผนดำเนินงาน	2568				2569				
		ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย	พ.ค.
1	ทบทวนทฤษฎีและจัดทำเอกสาร Proposal (บทที่ 1-3)									
2	กำหนด Requirements & Success Criteria II และ									
3	พัฒนา Prototype-A (เชื่อมต่อ ASR, LLM, และ Dice									
4	พัฒนา Rules Engine และ Game State Trackerพัฒนา Rules Engine และ Game									
5	พัฒนา UI (Character Sheet, Zone Map, Battle Log)									
6	บูรณาการระบบเป็น Prototype-B และทดสอบเบื้องต้น									
7	ทดลองใช้งานระบบ (Pilot Test) และรวบรวมผลตอบรับ									
8	ปรับปรุงระบบขั้นสุดท้าย และจัดทำรายงานฉบับสมบูรณ์ปรับปรุงระบบขั้น									

Project Timeline & Status

1. Current Phase (Oct - Dec 2025):

- **Goal:** Architecture Design & Prototype-A Development.
- **Status: On Track.** 
 - **Completed:** System Architecture Design.
 - **In Progress:** Core Rules Engine Implementation.

2. Next Phase (Jan - Mar 2026):

- Full UI Development & Game State Integration.

Implementation Details (The Heart)

```

/media/mudmini009/WORK_UNIVE
- .dart_tool
- .idea
- build
- lib
- logic
  - abilities.dart
  - dice_roller.dart
  - rules_engine.dart
- models
  - character.dart
  - game_models.dart
  - game_state.dart
  - toon_converter.dart
- router
- services
- main.dart
- linux
- test
- ui
- .gitignore
- .metadata
- ai_dungeon_master.iml
- analysis_options.yaml
- LITE_5E_RULES.md
- pubspec.lock
- pubspec.yaml
- README.md

```

Project Structure

```

class Character {
  final String id;
  final String name;
  final String role;
  int hp;
  final int maxHp;
  final int ac;
  final Map<Stat, int> stats;
  Zone zone;
  List<String> inventory;
  Condition condition;

  Character({
    required this.id,
    required this.name,
    required this.role,
    required this.hp,
    required this.maxHp,
    required this.ac,
    required this.stats,
    this.zone = Zone.NEAR,
    List<String>? inventory,
    this.condition = Condition.NORMAL,
  }) : inventory = inventory ?? [];

  void takeDamage(int amount) {
    hp -= amount;
    if (hp <= 0) {
      hp = 0;
      condition = Condition.UNCONSCIOUS;
    } else if (hp < maxHp) {
      condition = Condition.INJURED;
    } else {
      condition = Condition.NORMAL;
    }
  }

  @override
  String toString() {
    return 'Character(id: $id, name: $name, r
  }
}

```

class Character

```

// Returns a Map with details of the attack.
static Map<String, dynamic> resolveAttack(
  Character attacker, Character target,
  {String attackType = 'melee'}) {
  // Step 1: Validate Target
  if (target.condition == Condition.DEAD ||
    target.condition == Condition.UNCONSCIOUS) {
    return {
      'isHit': false,
      'message': 'Target is dead or unconscious!',
      'roll': 0,
      'total': 0,
      'damage': 0,
      'isCrit': false,
      'disadvantage': false,
    };
  }

  // Step 2: Calculate Zone Distance
  int distance = (attacker.zone.index - target.zone.index).abs();

  if (attackType == 'melee' && distance > 0) {
    return {
      'isHit': false,
      'message': 'Target is out of melee range!',
      'roll': 0,
      'total': 0,
      'damage': 0,
      'isCrit': false,
      'disadvantage': false,
    };
  }

  // Step 3: Determine Stat & Damage Dice (Class Specifics)
  Stat attackStat = Stat.PHYS;
  int damageDiceSides = 6;
  int damageDiceCount = 1;
  int weaponBonus = 0;

  switch (attacker.role.toLowerCase()) {
    case 'fighter':
      attackStat = Stat.PHYS;
      damageDiceSides = 8; // Longsword
      break;
    case 'paladin':
      attackStat = Stat.PHYS;
      damageDiceSides = 6;
      damageDiceCount = 2; // Greatsword (2d6)
      break;
    case 'cleric':
      attackStat = Stat.PHYS;
      damageDiceSides = 8; // Warhammer
  }
}

```

resolveAttack

Implementation: Core Logic & Algorithm

1. Data Layer: Strict Typing Architecture

- **Implemented** lib/models/game_models.dart using Enums (Zone, Stat, Condition) to enforce strict data validation.
- Why? Prevents "Magic String" errors and ensures that states like DEAD or UNCONSCIOUS are handled globally.

2. Logic Layer: The resolveAttack() Algorithm

- **Implemented a 6-Step Deterministic Process in rules_engine.dart:**
 - Validation:** Check if Target is DEAD or UNCONSCIOUS.
 - Distance Calculation:** Use (Zone A - Zone B).abs() to validate Melee Range.
 - Polymorphism:** Switch-Case logic to assign correct Dice (d8 vs 2d6) based on Class Role.
 - Hit Calculation:** RNG(d20) + Stat Mod vs Target AC.
 - Damage Logic:** Applies Critical Hit rules (Double Dice) automatically.
 - State Update:** Directly modifies target.hp and updates Condition.

Verification (Detailed Unit Testing)

```
mudmini009@mudmini009-ASUS-TUF-Gaming-A15 /media/mudmini009/WORK_UNIVERSITY/UNIVERSITY/YR4/FRA461
#0: Loading test/comprehensive_rules_test.dart
#0: Comprehensive Rules Engine Tests 1. Core Mechanics Attack Roll should include Stat Modifier
#1: Comprehensive Rules Engine Tests 1. Core Mechanics Attack Hit vs AC Logic
#2: Comprehensive Rules Engine Tests 1. Core Mechanics Natural 20 should be Critical Hit (Auto Hit)
#3: Comprehensive Rules Engine Tests 1. Core Mechanics Damage Application reduces HP
#4: Comprehensive Rules Engine Tests 2. Class-Specific Logic Fighter uses PHYS and d8
#5: Comprehensive Rules Engine Tests 2. Class-Specific Logic Paladin uses PHYS and 2d6
#6: Comprehensive Rules Engine Tests 2. Class-Specific Logic Mage uses MENT and d10 for Spells
#7: Comprehensive Rules Engine Tests 3. Zone & Range Logic Melee at Distance 0 is Allowed
#8: Comprehensive Rules Engine Tests 3. Zone & Range Logic Melee at Distance 1 is Blocked
#9: Comprehensive Rules Engine Tests 3. Zone & Range Logic Ranged at Distance 2 has Disadvantage
#10: Comprehensive Rules Engine Tests 4. Edge Cases Cannot attack Dead target
#11: Comprehensive Rules Engine Tests 4. Edge Cases Case Insensitive Role
#12: All tests passed!
mudmini009@mudmini009-ASUS-TUF-Gaming-A15 /media/mudmini009/WORK_UNIVERSITY/UNIVERSITY/YR4/FRA461
```

Edge Case: Attack Dead Target -> BLOCKED

Zone Logic: Melee at Distance 1 -> FAIL

Probability: Fighter Dmg Range [1-8] -> VERIFIED

Verification: 12 Scenarios Unit Testing

1. Verification Strategy:

- Adopted a "Boundary Testing" approach to ensure the Engine handles edge cases correctly before AI Integration.

2. Critical Test Groups:

- Edge Cases:** Verified that attacks on DEAD targets return isHit: false immediately.
- Zone Logic:** Confirmed that Melee Attacks across different Zones (Distance > 0) are strictly Blocked by the Engine.
- Class Mechanics:**
 - Fighter:** Verified damage output stays within 1d8 range.
 - Paladin:** Verified damage scales correctly with 2d6.
- Accuracy:** Verified that Attack Roll + Stat >= AC results in a HIT, while lower values result in a MISS.

3. Result:

- 100% Pass Rate. The system acts as a "Firewall" against invalid game states.

Conclusion

Conclusion & Next Steps

🎨 Design: Complete ✓

🧠 Architecture: Validated ✓

⚙️ Core Logic:
Foundation Ready 🚧 ⚙️*



Phase 5:
Implementing
Abilities & AI
Integration

Current Status Summary

1. Design Completed:

- UI is finalized for Casual & Single Device play.

2. Architecture Validated:

- The "Hybrid-Arbitrated" model is finalized and backed by research.

3. Implementation Status:

- **Core Mechanics:** Attack Rolls, Checks, and Zone Logic are Verified.
- **In Progress:** Currently implementing specific Class Abilities (e.g., Smite, Spells) on top of the core engine.

Next Step:

- Finalize Class Abilities & Integrate with Gemini Flash-Lite for Prototype-A.



Contact Us

Institute of Field Robotics
King Mongkut's University of Technology Thonburi

A Cradle of Future Leaders in Robotics